

Uncertainty-Calibrated Diabetic Retinopathy Grading with Counterfactual Visual Explanations

Use of Modern Tools and Technologies

Gayathri · Cohort E02 · Department of Computer Science and Engineering · 18 September 2026 · repository commit `ec6e161`

1. Technology stack overview

The project deliberately uses current, actively maintained tooling across five concerns: deep learning, configuration and experiment management, uncertainty/explainability libraries, deployment runtimes and software-engineering quality gates. All versions below are pinned in `pyproject.toml` (compatible-release specifiers) and locked in `uv.lock`, so the environment is reproducible with one command.

Table 1 — Tools and technologies (versions as installed in the locked environment).

tool	version	role in the project	where it is used
PyTorch	2.14	Tensor computation, autograd, MPS/CUDA back-ends	All model, UQ and counterfactual code
Lightning	2.6	Training loop, checkpointing, early stopping, mixed precision	<code>dr_uq/models/grading_model.py</code> , <code>scripts/train.py</code>
timm	1.0	ImageNet-pretrained ResNet-50, EfficientNet-B4, ViT-B/16, Swin-T with variable input size and gradient checkpointing	<code>dr_uq/models/backbones.py</code>
Hydra + OmegaConf	1.3 / 2.3	Hierarchical config groups, command-line overrides, multirun sweeps, experiment presets	<code>configs/</code> , every script
Albumentations	2.0	Config-declared augmentation (flips, $\pm 180^\circ$ rotation, scale, brightness/contrast)	<code>dr_uq/data/datamodule.py</code>
OpenCV (headless)	5.0	FOV crop, Gaussian blur for Graham normalisation, resize, Laplacian sharpness	<code>dr_uq/data/preprocess.py</code> , <code>quality.py</code>
torchmetrics	1.9	QWK, accuracy, per-grade sensitivity/specificity, referable-DR AUROC during training	<code>dr_uq/models/grading_model.py</code>
NumPy / pandas / scikit-learn / SciPy	2.4 / 3.0 / 1.9	Manifests, metric implementations, AUROC, bootstrap resampling	<code>dr_uq/evaluation</code> , <code>dr_uq/selective</code>
netcal	1.4	Independent reference implementation used to cross-check the in-house ECE	<code>tests/unit/test_uncertainty.py</code>
Captum	0.9	Layer Grad-CAM attributive baseline	<code>dr_uq/counterfactual/validate.py</code>
LPIPS	0.1.4	Perceptual distance in inversion and counterfactual objectives	<code>dr_uq/counterfactual/invert.py</code>

tool	version	role in the project	where it is used
pytorch-fid	0.3	Fréchet Inception Distance for counterfactual plausibility	dr_uq/counterfactual/validate.py
StyleGAN2-ADA (NVLabs)	official	High-fidelity fundus generator for counterfactual synthesis (wrapped; placeholder fallback)	dr_uq/counterfactual/generator.py
ONNX / ONNX Runtime	1.22 / 1.30	Portable inference graph with the fitted temperature baked in; CPU deployment	dr_uq/deploy/export_onnx.py
TensorRT	10.x (pinned in Docker)	FP16 and INT8 engines with entropy calibration for edge GPUs	dr_uq/deploy/build_trt.py
Docker (NVIDIA PyTorch container)	25.06	Reproducible GPU serving/benchmark image	Dockerfile
Weights & Biases	0.30	Optional experiment tracking (offline by default)	dr_uq/utils.py
DVC	yaml stages	Data and split-manifest lineage	dvc.yaml
uv	0.11	Fast, lock-file based environment management (Python 3.11)	pyproject.toml, uv.lock
ruff · black · mypy	0.16 · 26.5 · 2.3	Linting/import order, formatting, static typing of the whole package	pyproject.toml, .pre-commit-config.yaml
pytest	9.1	50 test functions (59 items) — unit + integration	tests/
pre-commit · GitHub Actions	4.6 · v4 actions	Local hooks and CI pipeline (lint, format, types, config check, tests)	.github/workflows/ci.yml

2. How the key technologies are used

2.1 PyTorch 2 + Lightning + timm

Lightning provides the training loop, checkpointing on validation QWK, early stopping, learning-rate monitoring and mixed precision, while the model recipe lives in one `LightningModule`. `timm` supplies ImageNet-pretrained CNN and transformer backbones; ViT and Swin accept a 512^2 input directly (positional embeddings are interpolated, Swin uses window 8) and gradient checkpointing is switched on for ViT from configuration. The training script falls back from 16-bit mixed precision to 32-bit on CPU/Apple-silicon so the same config runs everywhere.

dr_uq/models/backbones.py

```
def build_backbone(cfg: DictConfig) -> GradingModel:
    """Create a :class:`GradingModel` from a ``model`` config group.

    Args:
        cfg: Config with ``timm_name``, ``pretrained``, ``dropout``, ``num_classes``,
            ``img_size``, ``grad_checkpointing`` and (Swin) ``window_size``.

    Returns:
        A :class:`GradingModel` with dropout before the head on every architecture.
    """
    timm_name = str(cfg.get("timm_name") or BACKBONES[str(cfg.name)])
    backbone = timm.create_model(
        timm_name,
```

```

        pretrained=bool(cfg.get("pretrained", True)),
        num_classes=0,
        **_timm_kwargs(cfg),
    )
    set_ckpt = getattr(backbone, "set_grad_checkpointing", None)
    if bool(cfg.get("grad_checkpointing", False)) and callable(set_ckpt):
        set_ckpt(True)
        log.info("gradient checkpointing enabled for %s", timm_name)
    return GradingModel(
        backbone=backbone,
        num_features=int(backbone.num_features), # type: ignore[arg-type]
        num_classes=int(cfg.get("num_classes", 5)),
        dropout=float(cfg.get("dropout", 0.2)),
    )

```

2.2 Hydra configuration groups and multirun

Every choice is a config group option; the primary `config.yaml` composes `data`, `model`, `uq`, `train`, `deploy` and an optional `experiment` preset. Hydra's `multirun` launches factorial sweeps in one command, and a CI check composes every option to catch broken interpolations early.

```

# one training run / a 3 x 3 factorial sweep / a named experiment
python scripts/train.py data=aptos model=resnet50 train.seed=0
python scripts/train.py -m model=resnet50,efficientnet_b4,vit_b16 train.seed=0,1,2 data=aptos
python scripts/sweep.py experiment=calib_sweep          # train, evaluate (x4 UQ), aggregate
with CIs

```

`configs/config.yaml`

```

defaults:
  - data: synthetic
  - model: resnet50
  - uq: none
  - train: default
  - deploy: default
  - _self_
  - experiment: null

project: dr_uq
run_name: ${data.name}-${model.name}-s${train.seed}
run_dir: runs/${run_name}

```

2.3 Uncertainty and explainability libraries

Calibration metrics are implemented in-house for transparency and verified against **netcal**; **Captum** provides the Grad-CAM attributive baseline; **LPIPS** and **pytorch-fid** supply the perceptual and distributional measures for counterfactual quality; the official **StyleGAN2-ADA** networks are wrapped behind a small interface with a CPU-testable placeholder.

`tests/unit/test_uncertainty.py`

```

def test_ece_matches_netcal() -> None:
    netcal_metrics = pytest.importorskip("netcal.metrics")
    rng = np.random.default_rng(0)
    logits = rng.normal(size=(500, 5)) * 2
    probs = np.exp(logits) / np.exp(logits).sum(1, keepdims=True)
    labels = rng.integers(0, 5, 500)
    ours = expected_calibration_error(probs, labels, 15)
    theirs = float(netcal_metrics.ECE(bins=15).measure(probs, labels))
    assert ours == pytest.approx(theirs, abs=1e-6)

```

2.4 Deployment runtimes: ONNX Runtime, TensorRT, Docker

The grader and its fitted temperature are exported as a single ONNX graph with a dynamic batch axis, and ONNX Runtime output is asserted to match PyTorch. TensorRT engines (FP16, and INT8 with an entropy calibrator fed by validation batches and a persisted calibration cache) target edge GPUs; all TensorRT code is imported lazily so the package installs and tests on machines without a GPU. A Dockerfile based on the NVIDIA PyTorch container pins the TensorRT version for reproducible benchmarks.

dr_uq/deploy/export_onnx.py

```
def export_onnx(
    model: GradingModel,
    path: Path,
    temperature: float = 1.0,
    img_size: int = 512,
    opset: int = 17,
    atol: float = 1e-3,
    verify: bool = True,
) -> Path:
    """Export ``softmax-ready`` temperature-scaled logits ``f(x)/T`` to ONNX.

    Args:
        model: Trained grader.
        path: Output ``.onnx`` path.
        temperature: Fitted temperature baked into the graph.
        img_size: Input side length used for the example input (dynamic batch axis).
        opset: ONNX opset version.
        atol: Absolute tolerance for the PyTorch-vs-ONNX Runtime check.
        verify: Run the check (requires ``onnxruntime``).

    Returns:
        The written path.

    Raises:
        AssertionError: If outputs differ by more than ``atol``.
    """
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    wrapped = TemperatureScaled(model, temperature).eval().cpu()
    wrapped.model.dropout_active = False
    example = torch.randn(1, 3, img_size, img_size)
    with torch.no_grad():
        torch.onnx.export(
            wrapped,
            (example,),
            str(path),
            input_names=["image"],
            output_names=["logits"],
            dynamic_axes={"image": {0: "batch"}, "logits": {0: "batch"}},
            opset_version=opset,
            dynamo=False,
        )
    log.info("exported ONNX to %s (T=%.4f)", path, temperature)
    if verify:
        verify_onnx(wrapped, path, example, atol=atol)
    return path
```

2.5 Engineering quality gates

uv creates the Python 3.11 environment from the lock file in seconds. **ruff** (pyflakes, pycodestyle, isort, bugbear), **black** (line length 100) and **mypy** (type hints on every public function) run locally through **pre-commit** and on every push through **GitHub Actions**, together with **pytest** and a Hydra composition

check. **DVC** stages document data and manifest lineage; **Weights & Biases** is available for tracking but defaults to offline mode so that no data leaves the machine unless requested.

`.github/workflows/ci.yml`

```
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 45
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v3
        with:
          enable-cache: true
      - name: Install
        run: |
          uv python install 3.11
          uv sync --extra dev --frozen
          uv pip install -e .
      - name: Ruff
        run: uv run ruff check .
      - name: Black
        run: uv run black --check .
      - name: Mypy
        run: uv run mypy
      - name: Hydra config composition check
        run: |
          uv run python scripts/check_configs.py
      - name: Tests
        env:
          DR_UQ_TEST_FAST: "1"
        run: uv run pytest -q
```

3. Hardware and platforms

Development and the preliminary runs used an Apple M4 Max (Metal/MPS back-end, 48 GB unified memory) — the code selects CUDA, MPS or CPU automatically. GPU-specific paths (16-bit mixed precision, TensorRT engines, INT8 calibration) are configured and guarded for an NVIDIA workstation/cloud GPU, where the full APTOS/EyePACS sweeps will run.

Why these choices

- **timm + Lightning** give access to CNN and transformer families through one interface, which is required to compare calibration across architecture families (RQ1).
- **Hydra** makes the 36-run factorial a one-line command and ties every number to a config name.
- **ONNX + TensorRT** are the standard path to low-latency inference on inexpensive edge GPUs used in screening camps (RQ5).
- **ruff/black/mypy/pytest/CI** keep a 6 000-line research codebase maintainable by one person and reviewable by others.